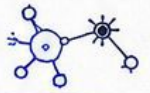


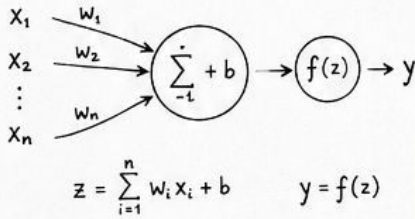


DEEP LEARNING CHEAT SHEET



1. NEURAL NETWORK BASICS

- Neuron : Basic unit of NN
- Computes weighted sum + bias
- Passes through activation function



Where,

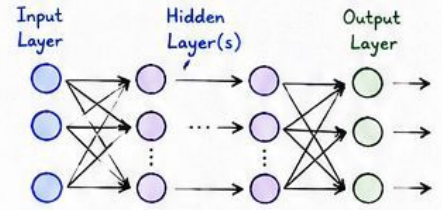
x_i = input , w_i = weight , b = bias

z = weighted sum , y = output

2. ACTIVATION FUNCTIONS

- Sigmoid : $\sigma(z) = \frac{1}{1+e^{-z}}$
- Tanh : $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- ReLU : $f(z) = \max(0, z)$
- Leaky ReLU : $f(z) = \max(\alpha z, z)$ (α small, e.g. 0.01)
- Softmax (for output layer): $\sigma(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ (outputs probabilities)

3. LAYERS IN NEURAL NETWORK



- Input Layer : receives features
- Hidden Layer(s) : learns representations
- Output Layer : makes predictions

4. LOSS FUNCTIONS

- Mean Squared Error (Regression)
 $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- Binary Cross Entropy (Binary Classification)
 $BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$
- Categorical Cross Entropy (Multi-class)
 $CCE = -\sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$

5. OPTIMIZERS

- Gradient Descent
 $\theta = \theta - \eta \nabla L(\theta)$
- Momentum
 $v_t = \beta v_{t-1} + (1-\beta) \nabla L(\theta)$
 $\theta = \theta - \eta v_t$
- RMSprop
 $s_t = \beta s_{t-1} + (1-\beta) (\nabla L(\theta))^2$
 $\theta = \theta - \frac{\eta}{\sqrt{s_t + \epsilon}} \nabla L(\theta)$
- Adam (Most popular)
Combines Momentum + RMSprop
(Fast convergence, stable)

6. BACKPROPAGATION

1. Forward pass : compute output $\hat{y}$
2. Compute loss $L(\hat{y}, y)$
3. Backward pass : compute gradients using chain rule $(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}, \dots)$
4. Update parameters (weights & biases) using optimizer

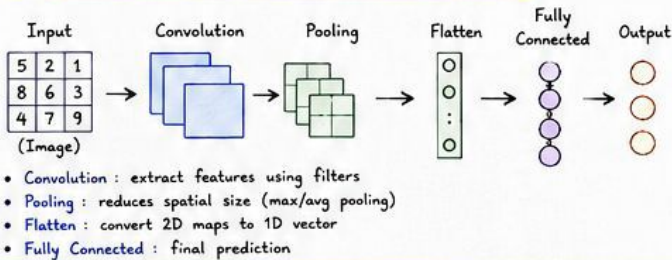
Chain Rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

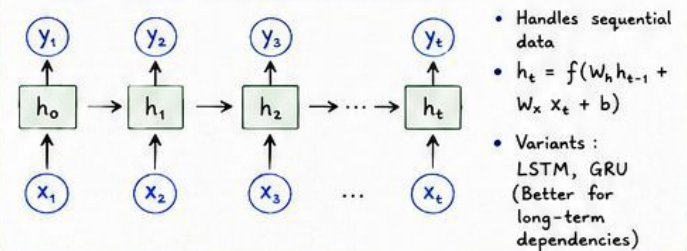
7. REGULARIZATION TECHNIQUES

- L2 Regularization (Ridge)
 $L' = L + \lambda \sum w_i^2$
- Dropout
Randomly drop neurons during training (prevents overfitting)
- Early Stopping
Stop training when validation loss stops improving

8. CONVOLUTIONAL NEURAL NETWORK (CNN)



9. RECURRENT NEURAL NETWORK (RNN)



10. COMMON METRICS

- Classification
Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$
Precision = $\frac{TP}{TP + FP}$
Recall = $\frac{TP}{TP + FN}$
F1 Score = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
- Regression
MAE = $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
MSE = $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
RMSE = $\sqrt{MSE}$
 $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

11. HYPERPARAMETERS

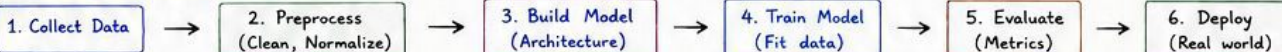
- Learning Rate (η)
- Batch Size
- Number of Layers
- Number of Neurons
- Activation Function
- Optimizer
- Dropout Rate
- Epochs

Tune these to get the best performance!

12. TIPS FOR TRAINING

- ✓ Normalize your data
- ✓ Start simple, then increase complexity
- ✓ Use appropriate activation functions
- ✓ Monitor training & validation loss
- ✓ Prevent overfitting (Dropout, L2, Early stopping)
- ✓ Use GPUs for faster training
- ✓ Experiment & keep learning!

13. DEEP LEARNING WORKFLOW



14. POPULAR ARCHITECTURES

- LeNet - Early CNN
- AlexNet - Deep CNN
- VGGNet - Simple & deep CNN
- ResNet - Residual connections
- Inception - Multi-scale features
- LSTM / GRU - For sequences
- Transformer - Attention based

15. POPULAR LIBRARIES

- TensorFlow / Keras
- PyTorch
- JAX
- NumPy
- Pandas
- OpenCV

16. DATA PREPROCESSING

- Handle missing values
- Normalize / Standardize
- Encode categorical data
- Train / Validation / Test split
- Data Augmentation (for images)
- Feature Engineering

17. OVERFITTING vs UNDERFITTING

- | Overfitting | Underfitting |
|--|---|
| • Model too complex | • Model too simple |
| • Low training error | • High training error |
| • High validation error | • High validation error |
| • Fix : More data, Regularization, Dropout, Early stopping | • Fix : Increase model complexity, Train longer |



PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

"From Tensors to Real-World Impact"

Part 1/6

Basics & Core Concepts

1. What is PyTorch?

- Open-source deep learning framework by Meta (Facebook)
- Dynamic computation graph (easy to debug, Pythonic)
- Widely used in research & industry
- Great for prototyping → production



"Tensors and dynamic graphs, all you need!"

Common Use Cases:

- Computer Vision
- NLP
- Speech & Audio
- Time Series
- Reinforcement Learning
- Generative AI (LLMs, Diffusion, etc.)

2. Tensors - The Building Block

- Tensor = multi-dimensional array (similar to NumPy, but with GPU support)
- Key properties: shape, dtype, device
- Operations are vectorized and fast

Create Tensors

```
import torch
x = torch.tensor([1, 2, 3])
print(x)           # tensor([1, 2, 3])
print(x.shape)     # torch.Size([3])
print(x.dtype)     # torch.int64
```

```
x = torch.randn(2, 3) # random tensor
x = torch.zeros(3, 3) # all zeros
x = torch.ones(2, 2)  # all ones
x = torch.arange(0, 10, 2) [0, 2, 4, 6, 8]
```

Tensor Dimensions (Shapes)

Scalar (0D)



torch.tensor(5)
shape: []

Vector (1D)



shape: [3]

Matrix (2D)



shape: [2, 3]

3D Tensor (3D)



shape: [2, 3, 4]

Tensor Operations

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([0.5, 0.5, 0.5])
print(a + b) # element-wise
print(a * 2) # scalar multiply
print(a @ b) # dot product
print(a.mean()) # mean
print(a.reshape(3, 1)) # reshape
```

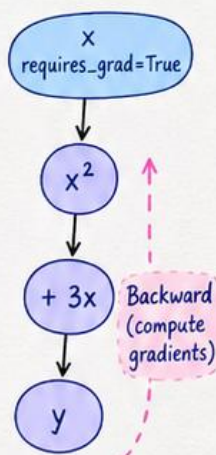
3. Computation Graph (Autograd)

- PyTorch builds a dynamic computation graph on the fly
- Automatic differentiation using autograd

Gradient computation

```
x = torch.tensor(2.0, requires_grad=True)
y = x**2 + 3*x
y.backward()
print(x.grad) # dy/dx = 2x + 3 = 7
```

.backward() computes gradients of the output w.r.t. all tensors with requires_grad=True



4. nn.Module - Building Models

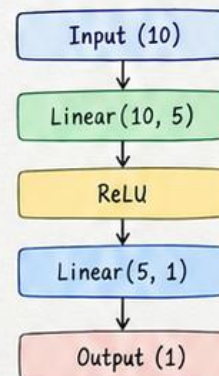
- All neural networks in PyTorch inherit from nn.Module
- Define layers in __init__ and forward pass in forward()

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(10, 5)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(5, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Model Structure



5. Loss Functions (nn)

- Measure how far predictions are from targets
- Choose based on problem type

Task Type	Common Loss	PyTorch Class
Regression	MSE	nn.MSELoss()
Binary Classification	Binary Cross Entropy	nn.BCEWithLogitsLoss()
Multi-class Classification	Cross Entropy	nn.CrossEntropyLoss()
Multi-label Classification	BCE	nn.BCEWithLogitsLoss()

```
criterion = nn.MSELoss()
loss = criterion(predictions, targets)
```

6. Optimizers

- Update model parameters using gradients
- Common: Adam (best general purpose)

Optimizer	Use Case
SGD	Simple, good baseline
Momentum	Faster convergence
Adam	Most popular (default)
AdamW	Better for Transformers
RMSprop	RNNs
Adagrad	Sparse data

```
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3
)
optimizer.zero_grad() # clear old grads
loss.backward()       # compute new grads
optimizer.step()      # update weights
```

7. Training Loop (Template)

- Core structure for training any model

```
num_epochs = 10
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for X, y in train_loader:
        y_pred = model(X)
        loss = criterion(y_pred, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
```

Next Part → Data Handling, Datasets & DataLoaders (Part 2/6)



Keep Learning. Keep Building.

Same tools. Bigger dreams.

Page 1 of 6



PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

"From Tensors to Trained Models"

Part 2/6

Data, Datasets
DataLoaders &
Transformations

8. Working with Data in PyTorch

- PyTorch provides easy tools to load, preprocess and feed data to models
- Main modules: torchvision, torch.utils.data
- Supports custom datasets and efficient loading with DataLoader

Key Components:

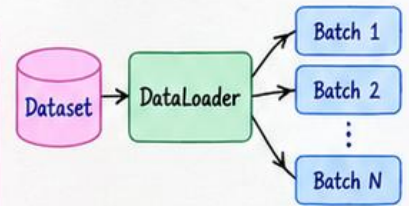
- Dataset – how to access data
- DataLoader – batch, shuffle, parallel loading
- Transforms – preprocessing/augmentation

9. Dataset and DataLoader

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

dataset = CustomDataset(X, y)
loader = DataLoader(dataset,
                    batch_size=32,
                    shuffle=True,
                    num_workers=4)
```



DataLoader handles:

- Batching
- Shuffling
- Parallel loading (num_workers)
- Works with custom datasets too!

10. Common Datasets (torchvision)

- Built-in datasets for vision tasks (image downloads and preprocessing handled for you)

```
import torchvision.datasets as dsets
import torchvision.transforms as transforms

transform = transforms.ToTensor()
train_set = dsets.CIFAR10(root='data',
                           train=True, download=True,
                           transform=transform)
train_loader = DataLoader(train_set,
                          batch_size=64, shuffle=True)
```

MNIST (28×28)



CIFAR-10 (32×32)



ImageNet (224×224)



Popular Datasets:

- MNIST – handwritten digits
- CIFAR-10/100 – small color images
- ImageNet – large scale dataset
- Custom datasets – use your own data

11. Transforms (Preprocessing & Augmentation)

- Transforms modify data before feeding to the model
- Useful for normalization, resizing, augmentation, etc.

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                        std=[0.5, 0.5, 0.5])
])
```

Common Transforms:

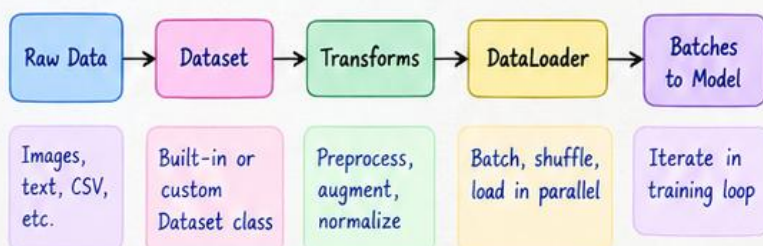
- Resize(), CenterCrop()
- RandomHorizontalFlip(), RandomRotation()
- ToTensor(), Normalize()
- ColorJitter(), RandomCrop(), RandomAffine()

Example Augmentations



Data augmentation helps improve generalization!

12. Data Pipeline (End-to-End)



13. Handling Different Data Types

Data Type	Common Use Case	Example / Notes
Images	Computer vision	torchvision.datasets + transforms
Text	NLP	Use tokenization (e.g. HuggingFace)
Tabular	Structured data	Use pandas + custom Dataset
Audio	Speech, sound	torchaudio
Time Series	Forecasting	Custom dataset, windowing
Graphs	Graph NN	PyTorch Geometric
Video	Video classification	Use 3D transforms / frame sampling

14. Best Practices for Data Loading

- ✓ Use DataLoader with appropriate batch size
- ✓ Shuffle training data
- ✓ Use data augmentation for better generalization
- ✓ Normalize input data
- ✓ Use multiple workers (num_workers > 0) for speed
- ✓ Handle class imbalance (e.g., WeightedRandomSampler)
- ✓ Verify data shapes and types
- ✓ Keep data pipeline modular and reproducible

Good data leads to great models!



15. Real-World Tips

- 💡 Start with small subset of data to test pipeline
- 💡 Visualize few samples after transforms
- 💡 Use consistent normalization (match train & test)
- 💡 For large datasets, consider streaming or distributed loading
- 💡 Use data augmentation strategically (not too heavy)
- 💡 For imbalanced data, use weighted sampling or focal loss
- 💡 Check data licensing when using public datasets
- 💡 Automate data download & preprocessing in scripts

"Data is the fuel for deep learning."



Clean Data. Better Models. Bigger Impact.

Prepare for real-world ML & AI roles.

Page 2 of 6

PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

"From Data to Intelligence"

Part 3/6

Neural Networks,
Training &
Evaluation

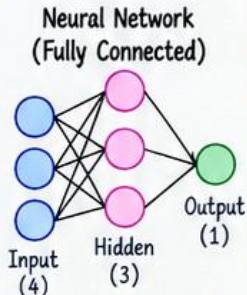
16. Neural Network Basics

- A neural network learns a function $f(x; \theta)$ composed of layers, activations and parameters.
- In PyTorch, build models by subclassing `nn.Module`

Basic Structure

```
import torch.nn as nn
class SimpleNN(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, out_dim)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```



Each layer applies a linear transformation followed by a non-linear activation function.

17. Common Layers in PyTorch

Layer	Description	Code Example
<code>nn.Linear()</code>	Fully connected layer	<code>nn.Linear(in, out)</code>
<code>nn.Conv2d()</code>	2D convolution layer (for images)	<code>nn.Conv2d(3, 32, 3)</code>
<code>nn.MaxPool2d()</code>	Max pooling (downsample)	<code>nn.MaxPool2d(2)</code>
<code>nn.BatchNorm2d()</code>	Batch normalization	<code>nn.BatchNorm2d(32)</code>
<code>nn.Dropout()</code>	Randomly zero activations (to prevent overfitting)	<code>nn.Dropout(0.5)</code>
<code>nn.Embedding()</code>	Embedding layer (for text/categorical data)	<code>nn.Embedding(vocab, dim)</code>
<code>nn.RNN / LSTM / GRU</code>	Recurrent layers (for sequences)	<code>nn.LSTM(input, hidden)</code>
<code>nn.Flatten()</code>	Flatten tensor	<code>nn.Flatten()</code>
<code>nn.Sequential()</code>	Stack layers sequentially	<code>nn.Sequential(...)</code>

18. Activation Functions

- Introduce non-linearity so the network can learn complex patterns.
- Applied element-wise.

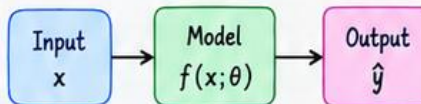
Function	Formula	PyTorch	Plot
ReLU	$f(x) = \max(0, x)$	<code>nn.ReLU()</code>	
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	<code>nn.Sigmoid()</code>	
Tanh	$\tanh(x)$	<code>nn.Tanh()</code>	
Leaky ReLU	$f(x) = \max(\alpha x, x)$	<code>nn.LeakyReLU()</code>	
Softmax	$\sigma(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	<code>nn.Softmax(dim=1)</code>	

ReLU is most common, but choice depends on the problem.

19. Forward Pass (Inference)

- Pass input through model to get output.
- No gradient computation needed.

```
model.eval() # set to evaluation mode
with torch.no_grad(): # no gradient tracking
    outputs = model(inputs)
    probs = torch.softmax(outputs, dim=1)
    preds = torch.argmax(probs, dim=1)
```



Use `model.eval()` and `torch.no_grad()` for inference (faster, less memory).

20. Loss Functions

- Measure how far predictions are from targets.
- PyTorch has many built-in loss functions.

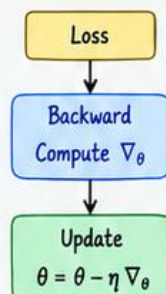
Task	Loss Function
Regression	<code>nn.MSELoss()</code> <code>nn.L1Loss()</code>
Binary Classification	<code>nn.BCEWithLogitsLoss()</code>
Multi-class Classification	<code>nn.CrossEntropyLoss()</code>
Multi-label Classification	<code>nn.BCEWithLogitsLoss()</code>
Other	<code>nn.KLDivLoss()</code> , <code>nn.HuberLoss()</code> , etc.

For classification, use raw logits with `CrossEntropyLoss` (no softmax needed).

21. Backpropagation (Training Step)

- Compute gradients using chain rule.
- Update model parameters using an optimizer.

```
# one training step
model.train()
optimizer.zero_grad() # clear old grads
outputs = model(inputs) # forward pass
loss = criterion(outputs, targets)
loss.backward() # compute gradients
optimizer.step() # update parameters
```



PyTorch builds a computation graph dynamically during the forward pass and uses it for backprop.

22. Training vs Evaluation Mode

Mode	Method	Behavior	Use Case
Training	<code>model.train()</code>	• Enable dropout • Update BatchNorm stats • Compute gradients	During training loop
Evaluation	<code>model.eval()</code>	• Disable dropout • Use running BatchNorm stats • No gradient computation (usually with <code>torch.no_grad()</code>)	Validation / Testing / Inference

Important: Always set the correct mode, otherwise results may be inconsistent!

23. Model Saving and Loading

- Save trained model weights for later use.
- Commonly save `state_dict` (rather than entire model).

```
# Save
torch.save(model.state_dict(), "model.pth")
# Load
model = SimpleNN(in_dim, out_dim)
model.load_state_dict(torch.load("model.pth"))
model.eval()
```



Tips:

- Save only `state_dict` (more flexible)
- Also save optimizer state (for resuming training)
- Keep track of model hyperparameters

PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

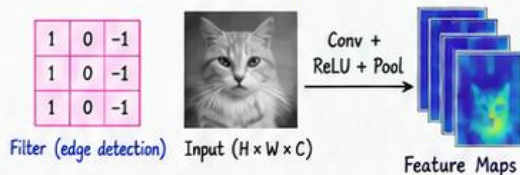
"From Models to Real-World Performance"

Part 4/6

CNNs, RNNs,
Attention &
Modern Architectures

24. Convolutional Neural Networks (CNNs)

- Great for grid-like data (images, video)
- Convolutions capture local patterns (edges, textures)
- Parameter sharing → fewer parameters



```
import torch.nn as nn
class CNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.fc = nn.Linear(64*8*8, num_classes)

    def forward(self, x):
        x = self.pool(nn.ReLU()(self.conv1(x)))
        x = self.pool(nn.ReLU()(self.conv2(x)))
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

25. Pooling Layers

- Reduce spatial dimensions
- Most common: Max Pooling, Avg Pooling
- Helps with translation invariance

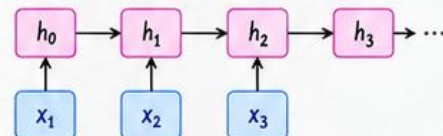


```
pool = nn.MaxPool2d(kernel_size=2)
x = pool(x) # reduces H, W by 2
```

Pooling keeps important information while reducing computation.

26. Recurrent Neural Networks (RNNs)

- Useful for sequential data (text, time series, speech)
- Maintain a hidden state
- Variants: RNN, LSTM, GRU (handle long-term deps)



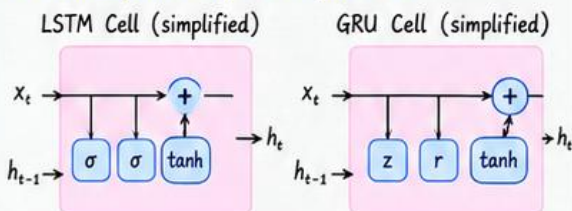
```
class RNNModel(nn.Module):
    def __init__(self, input_size, hidden_size,
                  num_layers=1, num_classes=2):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size,
                           num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x): # x: (B, T, F)
        out, h = self.rnn(x) # out: (B, T, H)
        out = self.fc(out[:, -1, :]) # last time step
        return out
```

Use LSTM/GRU for better long-term memory!

27. LSTM and GRU

- LSTM: uses gates (input, forget, output)
- GRU: simpler, fewer parameters
- Both help learn long-term dependencies

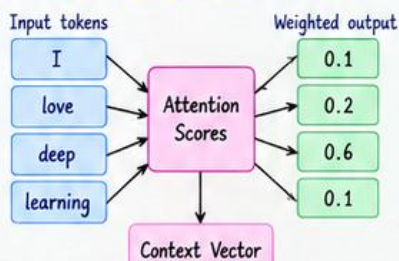


```
self.lstm = nn.LSTM(input_size, hidden_size,
                    num_layers=2, batch_first=True)
self.gru = nn.GRU(input_size, hidden_size,
                  num_layers=2, batch_first=True)
```

LSTM → better for complex sequences, GRU → faster & simpler.

28. Attention Mechanism

- Focus on important parts of the input
- Used in seq2seq models, Transformers
- Computes a weighted sum of values



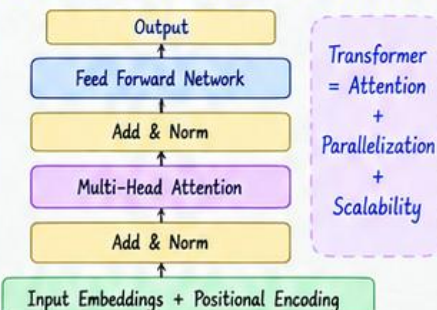
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

```
attn = nn.MultiheadAttention(embed_dim=512,
                             num_heads=8,
                             batch_first=True)
out, weights = attn(query, key, value)
```

Attention helps the model focus on what matters!

29. Transformers Overview

- Built on self-attention (no recurrence)
- Highly parallelizable
- Foundation of modern LLMs (BERT, GPT, etc.)



```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=512, nhead=8)
transformer = nn.TransformerEncoder(
    encoder_layer, num_layers=6)
out = transformer(src)
```

30. Popular Pretrained Models (Transfer Learning)

- Use pretrained models from torchvision or Hugging Face
- Fine-tune on your dataset (small data works well)

Using torchvision (ResNet)

```
import torchvision.models as models
model = models.resnet18(weights="DEFAULT")
model.fc = nn.Linear(model.fc.in_features, 10)
```

Common models: ResNet, VGG, EfficientNet, DenseNet, MobileNet

Using Hugging Face Transformers

```
from transformers import
AutoModelForSequenceClassification
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2)
```

Popular models: BERT, RoBERTa, GPT, T5, ViT (Vision Transformer), CLIP, LLaMA

Transfer learning = faster training + better performance!

31. Model Architectures Comparison

Model Type	Strengths	Use Cases	Example
CNN	Good for spatial data	Images, video	ResNet, EfficientNet
RNN	Sequential modeling	Text, time series	RNN, LSTM, GRU
Transformer	Attention, parallel	NLP, vision, audio	BERT, GPT, ViT
Autoencoder	Unsupervised learning	Dimensionality reduction	AE, VAE
GAN	Generative modeling	Image generation	DCGAN, StyleGAN
Diffusion Model	High-quality generation	Images, audio, video	Stable Diffusion
Graph NN	Graph structured data	Social networks, molecules	GCN, GAT

"Different architectures for different problems — choose the right tool!"



PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

"From Experiments to Real-World Applications"

Part 5/6

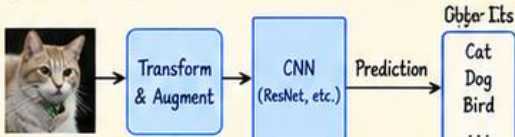
Computer Vision,
NLP, Generative AI
& Modern Trends

32. Computer Vision with PyTorch

- Common tasks: image classification, object detection, segmentation, image generation
- Use torchvision for datasets, transforms, pretrained models
- Data augmentation improves generalization

```
from torchvision import datasets, transforms, models
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])
dataset = datasets.CIFAR10(root='data', train=True,
    download=True, transform=transform)
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 10)
```

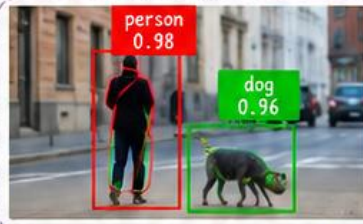
Typical CV Pipeline:



33. Object Detection (Faster R-CNN)

- Detect multiple objects with bounding boxes
- Use torchvision.models.detection
- Popular datasets: COCO, Pascal VOC

```
from torchvision.models.detection import
    fasterrcnn_resnet50_fpn
model = fasterrcnn_resnet50_fpn(weights='DEFAULT')
model.eval()
```

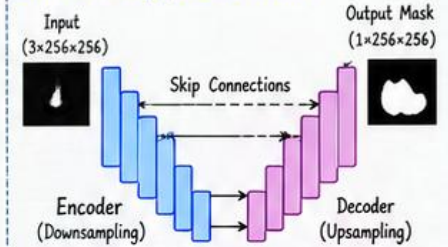


Object detection predicts:

- class label
- bounding box (x1, y1, x2, y2)
- confidence score

34. Image Segmentation (U-Net)

- Pixel-wise classification (e.g., medical, autonomous driving)
- U-Net is a popular architecture



```
class UNet(nn.Module):
    # simplified U-Net (details omitted)
    def __init__(self):
        super().__init__()
        # define encoder, decoder, skip connections
    def forward(self, x):
        # return segmentation mask
        return x
```

Applications: medical imaging, self-driving cars, remote sensing, image editing

35. Natural Language Processing (NLP)

- Use tokenizers to convert text to tokens
- Models: RNN/LSTM, Transformers (BERT, GPT, etc.)
- Use Hugging Face Transformers library

```
from transformers import AutoTokenizer, AutoModel
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

text = "Deep learning is amazing!"
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)
print(outputs.last_hidden_state.shape) # (1, seq_len, hidden)
```



36. Text Generation (GPT-style)

- Use pretrained language models
- Generate text with simple API

```
from transformers import AutoTokenizer,
    AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained(
    "gpt2")
model = AutoModelForCausalLM.from_pretrained(
    "gpt2")

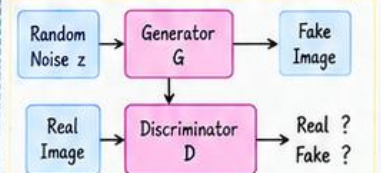
input_text = "The future of AI is"
inputs = tokenizer(input_text, return_tensors=
    "pt")
outputs = model.generate(**inputs, max_new_
    tokens=50)
print(tokenizer.decode(outputs[0],
    skip_special_tokens=True))
```

Used in chatbots, content generation, code generation, and more!

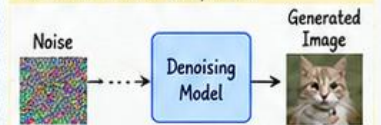
37. Generative Models

- Learn data distribution and generate new samples
- Popular models: Autoencoders, GANs, Diffusion Models

GAN (Generative Adversarial Network)



Diffusion Model (Simplified)



38. Transfer Learning & Fine-tuning

- Use pretrained models and fine-tune on your dataset
- Faster training, works well with small data

```
model = models.resnet18(weights='DEFAULT')
for param in model.parameters():
    param.requires_grad = False # freeze base
model.fc = nn.Linear(model.fc.in_features, num_classes)

# train only the final layer (or unfreeze some layers)
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Tips:

- ✓ Freeze early layers, fine-tune later layers
- ✓ Use lower learning rate
- ✓ Works great for CV, NLP and many other tasks

Leverage
pretrained
knowledge
!

39. Recent Trends & In-Demand Areas (2027+)

Trend / Area	Key Idea	Example Tools / Models	Real-World Roles
Large Language Models (LLMs)	Train, fine-tune, apply LLMs	GPT-4/5, Llama 3, Mistral, Gemma	LLM Engineer, AI Product Engineer
Multimodal AI	Text + Image + Audio	CLIP, LLaVA, GPT-4V	Multimodal AI Engineer
RAG (Retrieval Augmented Generation)	Combine search with LLMs	LangChain, LlamaIndex	RAG Engineer, AI Solutions Architect
AI Agents	Autonomous task execution	AutoGPT, CrewAI, LangGraph	AI Agent Developer
Edge AI / On-device ML	Deploy models on devices	TorchScript, ONNX, Quantization	Edge ML Engineer
Efficient Training	Larger models, lower cost	PEFT, LoRA, QLoRA, FSDP	ML Infrastructure Eng.
Responsible AI	Fairness, safety, alignment	RLHF, Red Teaming Tools	AI Ethics Specialist
Domain Applications	Healthcare, robotics, climate, finance, etc.	MONAI, HuggingFace, Custom models	AI Researcher, Domain AI Engineer

✦ These are high-growth areas for 2027 and beyond! ✦



New tools. New possibilities. Same PyTorch.

Build skills for the future.

Page 5 of 6



PyTorch + Deep Learning Cheat Sheet

Build · Train · Understand · Apply

"Same Tools. A Smarter You."

Part 6/6

Deployment, MLOps, Ecosystem, Career & Final Tips

40. Model Evaluation Metrics

- Choose metrics based on task (not just accuracy)
- Use torchmetrics or sklearn

Classification Metrics

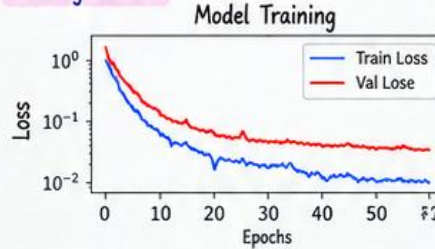
Metric	Formula/ Idea	Use Case
Accuracy	$\frac{TP + TN}{(TP + TN + FP + FN)}$	Balanced data
Precision	$\frac{TP}{(TP + FP)}$	High cost of false positives
Recall	$\frac{TP}{(TP + FN)}$	High cost of false negatives
F1-Score	$\frac{2 \cdot P \cdot R}{(P + R)}$	Imbalanced data
ROC-AUC	Area under ROC curve	Overall performance

For multi-class, use macro/micro averaging (e.g., average='macro' in sklearn).

41. Visualization in PyTorch

- Helps in debugging, understanding and presenting results
- Use matplotlib, seaborn, torchvision.utils

Training Curves

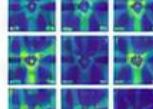


Show Images / Feature Maps

Original Image

Augmented

Feature Maps



```
from torchvision.utils import make_grid
import matplotlib.pyplot as plt
```

```
img_grid = make_grid(images, nrow=4)
plt.imshow(img_grid.permute(1, 2, 0))
plt.axis('off')
```

42. Saving, Deploying & Inference

- Take trained models to real-world applications
- Options: PyTorch, TorchScript, ONNX, etc.

Saving & Loading (Recap)

```
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))
model.eval()
```

TorchScript (for production)

```
scripted = torch.jit.script(model)
scripted.save('model_scripted.pt')
loaded = torch.jit.load('model_scripted.pt')
```

ONNX (for cross-platform deployment)

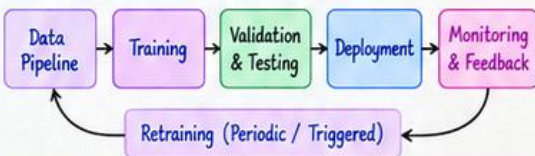
```
dummy_input = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, dummy_input,
                  'model.onnx',
                  input_names=['input'],
                  output_names=['output'])
```

Deployment Options



43. MLOps & Model Monitoring

- Keep models reliable, scalable and up-to-date
- Important for real-world ML systems



Key MLOps Tools

Tool	Use Case
MLflow	Experiment tracking, model registry
Weights & Biases	Visualization, tracking
DVC	Data & model versioning
Docker	Containerization
Kubernetes	Scalable deployment
Prometheus + Grafana	Monitoring
Airflow	Workflow orchestration

Monitor in Production

- Prediction latency
- Model performance (drift detection)
- Data drift
- Logging & alerting
- A/B testing
- Periodic retraining

MLOps turns ML models into reliable products!

44. PyTorch Ecosystem

- Many useful libraries built on top of PyTorch

Library	Purpose
torchvision	Datasets, transforms, pretrained models (vision)
torchaudio	Audio processing
torchtext	NLP utilities
torchmetrics	Evaluation metrics
pytorch-lightning	Simplified training loops
Hugging Face Transformers	Pretrained NLP, vision, multimodal models
timmm	State-of-the-art vision models
detectron2	Advanced computer vision (FAIR)
PyG (PyTorch Geometric)	Graph neural networks
accelerate	Easy multi-GPU / distributed training
bitsandbytes	8-bit training, efficient LLM training

Tip: Use the right tool instead of building everything from scratch!

45. Common Pitfalls & Debugging Tips

- Most deep learning problems are small mistakes!

- ✗ Forgetting model.train() / model.eval()
- ✗ Not moving data/model to GPU (device mismatch)
- ✗ Wrong tensor shapes (use print(x.shape))
- ✗ Not setting random seeds (results not reproducible)
- ✗ Exploding / vanishing gradients
- ✗ Overfitting (use regularization, more data, dropout)
- ✗ Data leakage (especially in time series)
- ✗ Ignoring data preprocessing / normalization
- ✗ Using wrong loss or activation for the task
- ✗ Not monitoring metrics (only looking at loss)

Debugging Tips

- ✓ Check tensor shapes at each step
- ✓ Start with a small batch and overfit (sanity check)
- ✓ Visualize data and predictions
- ✓ Use torch.autograd.set_detect_anomaly(True)
- ✓ Read error messages carefully (they are helpful!)
- ✓ Reduce complexity and isolate the issue
- ✓ Use prints, assertions, and visualization

46. Practice Projects (Build Your Skills)

- Best way to learn is by doing!
- Start simple → gradually increase difficulty

Level	Project Ideas
Beginner	Image classification (CIFAR-10), Digit recognition (MNIST)
Intermediate	Object detection, Image segmentation, Sentiment analysis, Text generation, Time series forecasting
Advanced	LLM fine-tuning, RAG-based Q&A system, Multimodal model (image + text), Recommendation system, Real-time app

Try to build and deploy at least 2-3 complete projects!

47. Key Takeaways & Next Steps

- ✓ PyTorch is flexible, powerful and widely used
- ✓ Understand the fundamentals deeply
- ✓ Practice with real datasets and projects
- ✓ Explore modern architectures (Transformers, Diffusion, etc.)
- ✓ Learn MLOps for real-world impact
- ✓ Keep up with new research and tools
- ✓ Build a portfolio (GitHub, blogs, demos)
- ✓ Contribute to open source
- ✓ Never stop learning!

"The best time to plant a tree was 10 years ago.
The next best time is now."

In-Demand Roles (2027+)

- AI / ML Engineer
- Deep Learning Engineer
- Applied Scientist
- LLM Engineer
- Computer Vision Engineer
- NLP Engineer
- MLOps Engineer
- AI Researcher
- Data Scientist (with DL)
- Multimodal AI Engineer
- Edge AI Engineer
- AI Product Manager

Build. Experiment. Share. Grow.
The future is open. 🚀



Keep Learning. Keep Building. A Smarter Future Awaits!

From concepts to creation — you've got this!